

Scanner 开发文档

目录

1. 获取条码	3
1. 设置输出方式为【广播模式】	3
2. 注册广播	3
3. 监听广播回调	3
4. 取消注册（将接收不到广播结果）	3
2. 接口	4
2.1 switchOutputMode【设置输出方式】	4
2.2 getOutputMode【获取输出方式】	4
2.3 startDecode【开始扫描】	4
2.4 stopDecode【停止扫描】	5
2.5 setTriggerMode【设置触发扫描模式】	5
2.6 getTriggerMode【获取触发扫描模式】	5
2.7 lockTriggler【禁用扫描】	6
2.8 unlockTriggler【启用扫描】	6
2.9 openScanner【打开扫描引擎】	6
2.10 closeScanner【关闭扫描引擎】	7
2.11 setParameterString【设置扫描参数（值为 String 类型）】	7
2.12 getParameterString【获取扫描参数（String 类型）】	8
2.13 setPropertyInts【设置扫描参数（值为 int 类型）】	8
2.14 getPropertyInts【获取扫描参数（int 类型）】	9
2.15.resetScannerParameters【重置扫描头相关设置】	9
3. 常用设置示例	10
4 获取扫描头图片（通过广播模式获取）	15
4.1 调用方法	16
4.2 广播方式接收图片数据	16
4.3 设置广播回调	16
4.4 取消注册	17
5 广播设置扫描参数(Android11 及以上版本支持)	18
广播设置扫码参数的 ACTION	18
5.1 扫描引擎开关	18
5.2 开始或停止扫描	19
5.3 设置输出模式	19
5.4 触发模式	19
5.5 锁定扫描	20
5.6 条码的广播设置	20
5.6.1 条码广播名称	20
5.6.2 条码键值名称	20
5.7 条码的广播模式下的声音和震动设置	21
5.7.1 声音设置	21
5.7.2 震动设置	21

1. 获取条码

说明：通过代码调用的方式，一般我们常用的输出模式是：广播模式，广播模式需要先通过 SDK 的 API 将 PDA 的输出模式设置成广播模式，具体操作如下：

1.1 设置输出方式为【广播模式】

```
ScanManager mScanManager = new ScanManager();  
mScanManager.switchOutputMode(0); //0: 广播模式 1: 键盘模式
```

1.2 注册广播

```
IntentFilter filter = new IntentFilter();  
filter.addAction(ScanManager.ACTION_DECODE);  
registerReceiver(mReceiver, filter);
```

1.3 监听广播回调

```
private BroadcastReceiver mReceiver = new BroadcastReceiver() {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        String barcodeStr = intent.getStringExtra(ScanManager.BARCODE_STRING_TAG  
); //barcodeStr 获取扫描的条码数据  
    }  
};
```

1.4 取消注册（将接收不到广播结果）

```
unregisterReceiver(mReceiver);
```

2. 接口

2.1 switchOutputMode 【设置输出方式】

定义	public boolean switchOutputMode(int mode);		
说明	设置 PDA 扫描到条码后的条码结果输出方式。		
参数	名称	类型	备注
	mode	int	0: 广播模式(Intent 输出) 1: 键盘模式
返回 (boolean)	true : 设置成功 false: 设置失败		
参考代码	mScanManager.switchOutputMode(0);		

2.2 getOutputMode 【获取输出方式】

定义	public int getOutputMode();		
说明	获取 PDA 当前使用的扫码结果输出方式		
参数	名称	类型	备注
返回 (int)	0: 广播模式(Intent 输出) 1: 键盘模式		
参考代码	mScanManager.getOutputMode();		

2.3 startDecode 【开始扫描】

定义	public boolean startDecode();		
说明	代码调起扫描头扫码（跟左右测扫码按键同等效果，如果使用按键触发扫描，可以不用调用该方法）		
参数	名称	类型	备注
返回 (boolean)	true : 设置成功 false: 设置失败		
参考代码	mScanManager.startDecode();		

2.4 stopDecode 【停止扫描】

定义	public boolean stopDecode();		
说明	代码控制停止扫描。		
参数	名称	类型	备注
返回 (boolean)	true : 设置成功 false: 设置失败		
参考代码	mScanManager.stopDecode();		

2.5 setTriggerMode 【设置触发扫描模式】

定义	public void setTriggerMode(TriggeringtgMode);		
说明			
参数	名称	类型	备注
	tgMode	Triggering	Triggering.PULSE 短接触发解码或超时停止 Triggering.CONTINUOUS 连扫模式 Triggering.HOST 长接触发, 松开停止
返回 (boolean)	true : 设置成功 false: 设置失败		
参考代码	mScanManager.setTriggerMode(Triggering.PULSE);		

2.6 getTriggerMode 【获取触发扫描模式】

定义	public Triggering getTriggerMode();		
说明			
参数	名称	类型	备注
返回 (Triggering)	Triggering.PULSE 短接触发解码或超时停止 Triggering.CONTINUOUS 连扫模式 Triggering.HOST 长接触发, 松开停止		

参考代码	<code>mScanManager.getTriggerMode();</code>
------	---

2.7 lockTriggler 【禁用扫描】

定义	<code>public boolean lockTriggler();</code>		
说明	设置扫描触发不活动(禁用扫描按钮和扫描方法)。		
参数	名称	类型	备注
返回	<code>true</code> : 设置成功		
(boolean)	<code>false</code> : 设置失败		
参考代码	<code>mScanManager.lockTriggler();</code>		

2.8 unlockTriggler 【启用扫描】

定义	<code>public boolean unlockTriggler();</code>		
说明	设置扫描触发激活(启用扫描按钮)。		
参数	名称	类型	备注
返回	<code>true</code> : 设置成功		
(boolean)	<code>false</code> : 设置失败		
参考代码	<code>mScanManager.unlockTriggler();</code>		

2.9 openScanner 【打开扫描引擎】

定义	<code>public boolean openScanner();</code>		
说明	打开扫描引擎（一般默认是开启状态，可以不用调用该方法）。		
参数	名称	类型	备注

返回	true：设置成功
(boolean)	false: 设置失败
参考代码	mScanManager.openScanner();

2.10 closeScanner 【关闭扫描引擎】

定义	public boolean closeScanner();		
说明	关闭扫描引擎，关闭后将无法扫描		
参数	名称	类型	备注
返回	true：设置成功		
(boolean)	false: 设置失败		
参考代码	mScanManager.closeScanner();		

2.11 setParameterString 【设置扫描参数（值为 String 类型）】

定义	public boolean setParameterString(int[] index, String[] value);		
说明	设置扫描参数		
参数	名称	类型	备注
	index	int[]	设置参数对应的参数 key，长度变长，可以传入任意长度的参数数组。
	value	String[]	设置对应参数的对应值，长度要与上面 key 的长度对应。
返回	true：设置成功		
(boolean)	false: 设置失败		
参考代码	mScanManager.setParameterString(new int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDG E_INTENT_DATA_STRING_TAG},new String[]{"com.pkuhit.intent.action.SCANRECVR"," 你 接 收 的		

	KeyString"}));
--	----------------

2.12 getParameterString【获取扫描参数（String 类型）】

定义	public String[] getParameterString(int[] index);		
说明	获取扫描参数		
参数	名称	类型	备注
返回 (String[])	String[] 结果的下标对应的就是传入的参数的下标参数的对应值。		
参考代码	<pre>int[] keys = new int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDG E_INTENT_DATA_STRING_TAG}; String[] ret = mScanManager.getParameterString(keys);</pre>		

2.13 setPropertyInts【设置扫描参数（值为 int 类型）】

定义	public boolean setPropertyInts(int[] index, int[] value);		
说明	设置扫描参数		
参数	名称	类型	备注
	index	int[]	设置参数对应的参数 key，长度变长，可以传入任意长度的参数数组。
	value	int[]	设置对应参数的对应值，长度要与上面 key 的长度对应。
返回 (boolean)	true：设置成功 false: 设置失败		

参考代码	<pre>mScanManager.setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE,PropertyID.SEN D_GOOD_READ_VIBRATE_ENABLE},new int[]{2,1});//// 打开广播模式 下的声音和震动</pre>
------	--

2.14 getPropertyInts 【获取扫描参数（int 类型）】

定义	public int[] getPropertyInts(int[] index);		
说明	获取扫描参数		
参数	名称	类型	备注
返回 (int[])	int[] 结果的下标对应的就是传入的参数的下标参数的对应值。		
参考代码	<pre>int[] keys = new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE,PropertyID.SEN D_GOOD_READ_VIBRATE_ENABLE}; int[] ret = mScanManager.getParameterString(keys);</pre>		

2.15.resetScannerParameters 【重置扫描头相关设置】

定义	public boolean resetScannerParameters();		
说明	将扫描头相关设置重置为出厂默认设置。		
参数	名称	类型	备注
返回 (boolean)	true : 设置成功 false: 设置失败		
参考代码	mScanManager.resetScannerParameters();		

3. 常用设置示例

不同模式下部分属性设置参考表		
模式	功能	参考代码
广播模式	开启或关闭声音和震动	<pre>ScanManager mScanManager = new ScanManager(); //以下的设置是针对扫描头的输出模式为 intent 方式，所以在设置属性的标识上有 SEND mScanManager.setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE},new int[]{0});// 声音 0 关闭 1 短促 2 尖锐 //扫描成功时震动 mScanManager.setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE},new int[]{0});//震动 0 关闭 1 开启</pre>
	设置广播模式的 action 和接收扫描结果的 keyString	<pre>new ScanManager().setParameterString(new int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDG E_INTENT_DATA_STRING_TAG},new String[]{"你的 Action","你接收 的 KeyString"});</pre>
键盘模式	开启或关闭键盘模式的声音和震动	<pre>ScanManager mScanManager = new ScanManager(); //以下的设置是针对扫描头的输出模式为键盘模式，所以在设置属性的标识上没有 SEND mScanManager.setPropertyInts(new int[]{PropertyID.GOOD_READ_BEEP_ENABLE},new int[]{0}); //声音 0 关闭 1 短促 2 尖锐 //扫描成功时震动 mScanManager.setPropertyInts(new int[]{PropertyID.GOOD_READ_VIBRATE_ENABLE},new int[]{0}); //震动 0 关闭 1 开启</pre>

	键盘类型	//输出模式为键盘模式 new ScanManager().setParameterInts(new int[] {PropertyID.WEDGE_KEYBOARD_TYPE},new int[] {1}); 0: 物理键盘 1: 输入法软件 2: 仅用物理键盘 3: 仅用输入法
	设置操作键 字符	new ScanManager().setParameterInts(new int[] {PropertyID.LABEL_APPEND_ENTER},new int[] {1}); //参数对应的可选择值 0 : None 1 : Carriage return 2 : IME action done 3 : TAB

常用功能参考表	
功能	参考代码
设置输出 模式	new ScanManager().switchOutputMode(0);//开启广播模式 new ScanManager().switchOutputMode(1);//开启键盘式
开启或关 闭应用标 识符	int [] keyInt = new int[] {PropertyID.LABEL_SEPARATOR_ENABLE}; int [] valueInt = new int[] {1};//0: 关闭 1: 开启 new ScanManager().setPropertyInts(keyInt,valueInt);
扫码附加 前后缀， 附加格式 0-6	new ScanManager(). setPropertyInts(new int[] {PropertyID.SEND_LABEL_PREFIX_SUFFIX},new int[] {1});
替换	new ScanManager() .setParameterString(new int[] {PropertyID.LABEL_MATCHER_TARGETREGEX,PropertyID.LABEL_MATCHER_R EPLACEMENT},new String[] {"oldStr","newStr"});
多条码设 置（不一 定每款设 备都会生 效）	int[] multiConfigbuf = new int[] { PropertyID.MULTI_DECODE_MODE, PropertyID.FULL_READ_MODE, PropertyID.BAR_CODES_TO_READ}; int[] multiConfigval = new int[3]; multiConfigval[0] = 1;//0 关闭多条码模式, 1 开启多条码模式 multiConfigval[1] = 1;//0 在一个或多个条形码被解码后生成一个解码事件 1:只有当至 少是解码到设置条形码的数量时，才生成回调到应用程序。 multiConfigval[2] = 4;//该参数设置解码个数， 1- 10 个条形码。默认值为 1。 new ScanManager().setParameterInts(multiConfigbuf, multiConfigval); 获取:

	//广播获取多条码扫描结果，需要将扫描头的输出模式设为广播输出。 <pre>String moreBarcodeStr=intent.getStringExtra("com.ubx.datawedge.data_string");</pre>
连扫模式下的去重设置	<pre>new ScanManager().setParameterInts(new int[]{PropertyID.DEC_Multiple_Decode_MODE},new int[]{0});</pre> //0：可重复解码 1：连续不重复解码 2：缓存内不重复解码
设置扫描到两个相同条码的间隔时间	<pre>//value * 100 ms ; MIN == 1 ,MAX == 99 设置扫描到两个相同条码的间隔时间，例如传 1--> 1*100ms == 100ms;传最大值 99 --> 99*100ms == 9900ms ,即间隔 9900 毫秒</pre> <pre>mScanManager.setPropertyInts(new int[]{PropertyID.TIMEOUT_BETWEEN_SAME_SYMBOL},new int[]{1});</pre>
设置扫描间隔，扫描超时时间	<pre>ScanManager mScanManager = new ScanManager(); mScanManager.setPropertyInts(new int[]{PropertyID.DEC_Multiple_Decode_INTERVAL},new int[]{50});</pre> //扫描间隔时间,默认 50ms (范围：0-5000ms) <pre>mScanManager.setParameterInts(new int[]{PropertyID.DEC_Multiple_Decode_TIMEOUT},new int[]{5000});</pre> //扫描超时时间,默认 5000ms (范围：50-60000ms)
打开或关闭相应的码制	<pre>ScanManager mScanManager = new ScanManager(); mScanManager.enableSymbology(Symbology.AZTEC,false);</pre> //AZTEC：具体的码制，false:不解码 true: 解码 <pre>mScanManager.enableAllSymbologies(true);</pre> //开启解码所有码制，true：开启 false：关闭
启动瞄准模式	<pre>mScanManager.setParameterInts(new int[]{PropertyID.DEC_PICKLIST_AIM_MODE},new int[]{1});</pre>
设置码校验级别默认为 1	<pre>int[] codeleveindex = new int[]{ PropertyID.LINEAR_CODE_TYPE_SECURITY_LEVEL}; int[] codelevevalue = new int[]{1};</pre> //级别等级 0-3. <pre>int ret2 = new ScanManager().setParameterInts(codeleveindex, codelevevalue);</pre>
设置打开或关闭码制和校验位	<pre>int[] codeChecksumKey = new int[]{ PropertyID.EAN13_ENABLE, //打开 EAN13 码制开关, 0: 关 1: 开 PropertyID.UPCA_ENABLE, //设置 UPCA 码制开关, 0: 关 1: 开 PropertyID.UPCE_ENABLE, //设置 UPCE 码制开关, 0: 关 1: 开 PropertyID.UPCE1_ENABLE, //设置 UPCE1 码制开关, 0: 关 1: 开 PropertyID.EAN13_SEND_CHECK, //设置 UPCE1 检验位开关, 0: 关 1: 开 PropertyID.UPCA_SEND_CHECK, //设置 UPCA 检验位开关, 0: 关 1: 开 PropertyID.UPCE_SEND_CHECK, //设置 UPCE 检验位开关, 0: 关 1: 开 PropertyID.UPCE1_SEND_CHECK};</pre> //设置 UPCE1 检验位开关, 0: 关 1: 开 <pre>int[] codeChecksumvalue = new int[]{1,1,1,1,0,0,0,0}; int ret = new ScanManager().setParameterInts(codeChecksumKey, codeChecksumvalue);</pre>
低对比图像模式	<pre>ScanManager mScanManager = new ScanManager(); mScanManager.setPropertyInts(new int[]{PropertyID.LOW_CONTRAST_IMPROVED,PropertyID.LOW_CONTRAST_IMPRO</pre>

	VED_ALGORITHM},new int[]{2,0}); //低对比图像模式，取值：0-5 //增强低对比度图像算法 0：关闭 1：打开
照明灯和瞄准灯设置	new ScanManager().setParameterInts(new int[]{PropertyID.DEC_2D_LIGHTS_MODE},new int[]{1}); //1：仅瞄准灯 2：仅照明灯 3：交替运行 4：同时运行
取景框	//DEC_2D_CENTERING_ENABLE 取值 0 1 关闭、开启 //DEC_2D_CENTERING_MODE 取景模式 0, 1, 2, 3 mScanManager.setParameterInts(new int[]{PropertyID.DEC_2D_CENTERING_ENABLE,PropertyID.DEC_2D_CENTERING_M ODE},new int[]{1,2}); //坐标 int[] key = new int[]{ PropertyID.DEC_2D_WINDOW_UPPER_LX, PropertyID.DEC_2D_WINDOW_UPPER_LY,PropertyID.DEC_2D_WINDOW_LOWER_R X,PropertyID.DEC_2D_WINDOW_LOWER_RY }; int[] value = new int[]{632, 346,640,354}; mScanManager.setParameterInts(key, value);
编码格式	int[] key = new int[]{ PropertyID.CODING_FORMAT};//中文编码格式 int[] value = new int[]{1};//0: UTF-8 1: GBK mScanManager.setParameterInts(key, value);
OCR 解码模式和 OCR 模板	int[] key = new int[]{ PropertyID.DEC_OCR_MODE,PropertyID.DEC_OCR_TEMPLATE }; //DEC_OCR_MODE:模式 (0-3) DEC_OCR_TEMPLATE: 模板 (0-4) int[] value = new int[]{3,0};//3:Both(模式) 0: User Defind (模板) int ret = mScanManager.setParameterInts(key, value);
OCR 自定义模板	int[] key = new int[]{ PropertyID.DEC_OCR_USER_TEMPLATE }; String[] action = { "1377777777777777770" }; boolean ret = mScanManager.setParameterString(key, action); 字符串开头 1 固定 第二个字符代表字体(1, 2, 3,4, 5) 3 代表 ocrA&ocrB 两种字体 第三个至倒数第二代表字符类型字符 (字符代表字符类型：数字字母空格组合，取值 5 (数字组合)，6 (字母组合)，7 (数字字母组合)，8. (任何支持的字符组合)) 字符串结尾 0 固定 比如 18 位的身份证： 1377777777777777770
优化一维码	ScanManager mScanManager = new ScanManager(); int[] key = new int[]{ PropertyID.FUZZY_1D_PROCESSING };//优化一维码 int[] value = new int[]{3 };//0:关闭 1: 开启 int ret = mScanManager.setParameterInts(key, value);
曝光参数设置	int[] key = new int[]{ PropertyID.IMAGE_EXPOSURE_MODE,PropertyID.DEC_ES_MAX_EXP,PropertyID .DEC_ES_MAX_GAIN, PropertyID.DEC_ES_TARGET_VALUE };//曝光参数设置

	<pre>int[] value = new int[] {1,500,4,4000 }; int ret = mScanManager.setParameterInts(key, value);</pre>
--	---

4 获取扫描头图片（通过广播模式获取）

本方法适用于获取扫描头上一次激光出光后解码的图片, 建议获取到条码后, 调用该方法。

4.1 调用方法

发送广播：

```
Intent intentImage = new Intent("action.scanner_capture_image");  
//intentImage.putExtra("rotate",180); //旋转图片角度  
sendBroadcast(intentImage);
```

4.2 广播方式接收图片数据

注册图片接收的广播：

```
String          action_scanner_capture_image_result          =  
"scanner_capture_image_result";  
  
IntentFilter filter = new IntentFilter();  
  
filter.addAction(action_scanner_capture_image_result);  
  
registerReceiver(mScanReceiver, filter);
```

4.3 设置广播回调

```
BroadcastReceiver mScanReceiver = new BroadcastReceiver() {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        if(intent          !=          null          &&  
intent.getAction().equals(action_scanner_capture_image_result)) {  
            byte[] bmpByte = intent.getByteArrayExtra("bitmapBytes");  
            if(bmpByte != null && bmpByte.length > 1) {  
                Bitmap          mBitmap          =  
BitmapFactory.decodeByteArray(bmpByte,          0,  
bmpByte.length).copy(Bitmap.Config.RGB_565, true);  
            }else {  
                Toast.makeText( ScanActivity.this, "获取图片失败 ",  
Toast.LENGTH_SHORT).show();  
            }  
        }  
    }  
}
```



```
        }  
    }  
}  
};
```

4.4 取消注册

如果不用再接收图片，可以取消注册。

```
unregisterReceiver(mScanReceiver);
```

5 广播设置扫描参数(Android11 及以上版本支持)

广播设置扫码参数的 ACTION

action	com.android.broadcaster.uscanner.settings
--------	---

示例：
Intent intent = new Intent("com.android.broadcaster.uscanner.settings");

5.1 扫描引擎开关

extra	keyInt
取值	int 型: -10
extra	valueInt
取值	int 型 0: 关闭 1: 开启

示例：
Intent intent = new Intent("com.android.broadcaster.uscanner.settings");
intent.putExtra("keyInt", -10);
intent.putExtra("valueInt", 1);
sendBroadcast(intent);

5.2 开始或停止扫描

extra	keyInt
取值	int 型: -11
extra	valueInt
取值	int 型 0: 关闭解码 1: 开始解码

示例:

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -11);
intent.putExtra("valueInt", 1);
sendBroadcast(intent);
```

5.3 设置输出模式

extra	keyInt
取值	int 型: -12
extra	valueInt
取值	int 型 0: 开启广播模式 1: 键盘模式 2: 广播模式+键盘模式

示例:

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -12);
intent.putExtra("valueInt", 0);
sendBroadcast(intent);
```

5.4 触发模式

extra	keyInt
取值	int 型: -13
extra	valueInt
取值	int 型 0: 自动模式（按下按键开始，超时后停止） 1: 连扫模式 2: 手动模式（按下按键后开始，松开按键或超时停止）

示例:

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
```

```
intent.putExtra("keyInt", -13);
intent.putExtra("valueInt", 0);
sendBroadcast(intent);
```

5.5 锁定扫描

extra	keyInt
取值	int 型: -14
extra	valueInt
取值	int 型 0: 锁定, 锁定后无法扫描 1: 解锁, 解锁后可以扫描

示例:

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -14);
intent.putExtra("valueInt", 1);
sendBroadcast(intent);
```

5.6 条码的广播设置

5.6.1 条码广播名称

extra	keyInt
取值	int 型: 200000 (PropertyID.WEDGE_INTENT_ACTION_NAME)
extra	valueStr
取值	String 型 默认是: "android.intent.ACTION_DECODE_DATA"

示例:

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 200000);
intent.putExtra("valueStr", "android.intent.ACTION_DECODE_DATA");
sendBroadcast(intent);
```

5.6.2 条码键值名称

extra	keyInt
取值	int 型: 200002 (PropertyID.WEDGE_INTENT_DATA_STRING_TAG)
extra	valueStr
取值	String 型 默认是: "barcode_string"

示例：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 200002);
intent.putExtra("valueStr","barcode_string");
sendBroadcast(intent);
```

5.7 条码的广播模式下的声音和震动设置

5.7.1 声音设置

extra	keyInt
取值	int 型： 6 (PropertyID.SEND_GOOD_READ_BEEP_ENABLE)
extra	valueInt
取值	int 型 0:无提示音 1: 短促 2: 尖锐

示例：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 6);
intent.putExtra("valueInt",1);
sendBroadcast(intent);
```

5.7.2 震动设置

extra	keyInt
取值	int 型： 7 (PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE)
extra	valueInt
取值	int 型 0:关闭震动 1: 开启震动

示例：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 7);
```

```
intent.putExtra("valueInt",1);  
sendBroadcast(intent);
```

备注：其他参数可参考 `PropertyID` 类或联系我司技术支持。